



PUSL3190 Computing Project

Project Proposal

Stock Price Indicator And Prediction Project

Supervisor: Diluka Wijesinghe

Name: Wakishta W Dissanayake

Plymouth Index Number: 10953506

Degree Program: BSc.(Hons) Computer Science

Contents

Chapter 01: Problem Statement	3
1.1 The Central Problem	3
1.2 The Gap in Accessibility and Performance.....	3
Chapter 02: Project Description.....	4
2.1 Project Objectives and Explanations.....	4
2.2 Project Scope	5
2.3 Project Keywords	6
Chapter 03: Research Gap	6
3.1 Review of Existing Solutions and Gaps	6
3.2 Justification of Project Differentiation.....	6
Chapter 04: Requirements Analysis.....	9
4.1 Functional Requirements	9
4.2 Non-Functional Requirements	9
4.3 Special Knowledge / Devices	10
Chapter 05: Finance	11
Chapter 06: External Organizations	12
Chapter 07: Time Frame	13
Chapter 08: Referencing	15

Chapter 01: Problem Statement

1.1 The Central Problem

The stock market is a highly dynamic and unpredictable environment, making accurate forecasting a significant challenge. Its behavior is influenced by numerous factors that change rapidly and interact in complex, non-linear ways. Traditional forecasting methods—such as ARIMA or simple machine learning models like Linear Regression often struggle to capture these intricate, long-term patterns, leading to unreliable predictions.

1.2 The Gap in Accessibility and Performance

In recent years, Deep Learning models, especially Long Short-Term Memory (LSTM) networks, have shown remarkable success in understanding and predicting complex time-series data like stock prices. However, these models are typically limited to specialized Python environments such as Jupyter notebooks, which are not easily accessible to everyday users.

As a result, individual investors and decision-makers often face several challenges:

- High-cost enterprise tools, which are out of reach for small investors or independent researchers.
- Outdated forecasting systems, which rely on older models and offer poor user experiences.

This project aims to bridge that gap by creating a modern, user-friendly web platform that combines the power of Python-based LSTM forecasting with the scalability and responsiveness of a Next.js frontend. By integrating advanced data science into an accessible web environment, this system empowers users to make more informed decisions using cutting-edge predictive insights.

Chapter 02: Project Description

2.1 Project Objectives and Explanations

The project aims to develop a robust, end-to-end stock price forecasting application delivered through a high-performance web interface.

Objective	Explanation (What/How)
O1: Develop a High-Accuracy LSTM Deep Learning Model	To design, train, and optimize a multivariate LSTM network using Python (TensorFlow/Keras). The model will incorporate historical price data, trading volume, and calculated technical indicators (e.g., RSI, MACD) to capture non-linear patterns and long-term dependencies in the financial time series data.
O2: Implement a Decoupled Microservice Architecture	To build a two-tiered system where the Python ML Model is deployed as an independent RESTful microservice (using Flask/FastAPI). This service will be callable by the Next.js application, ensuring that the intensive computational load of the model is decoupled from the web application layer.
O3: Design and Develop a Next.js User Interface	To utilize the Next.js framework to create a single-page, responsive, and performance-optimized frontend application that provides user authentication, ticker search functionality, and real-time interactive charting of predicted and historical prices.
O4: Evaluate Model Performance and Justify Superiority	To conduct rigorous model validation, comparing the LSTM model's performance against at least one conventional statistical model (e.g., ARIMA) using standardized metrics like Root Mean Square Error (RMSE) and Mean Absolute Error (MAE).

2.2 Project Scope

The scope of this project encompasses the end-to-end design, development, training, integration, and deployment of a stock price forecasting and visualization platform powered by deep learning. The system will integrate a Python-based LSTM (Long Short-Term Memory) model for prediction with a Next.js-based frontend to deliver a fast, interactive, and user-friendly web experience.

The project begins with data acquisition and preprocessing, which involves collecting historical stock data from reliable APIs like Yahoo Finance. This data will be cleaned, normalized, and converted into time-series sequences suitable for training the LSTM model. The system will also compute key technical indicators—including the Moving Average (MA), Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD)—to enhance the predictive capability of the model by capturing short-term and long-term market patterns.

Following data preparation, the LSTM deep learning model will be developed, trained, and validated using Python (TensorFlow). The model will be fine-tuned through experimentation with different hyperparameters, activation functions, and layer configurations to achieve the best balance between prediction accuracy and computational efficiency. To evaluate the model's performance, it will be compared against traditional forecasting methods such as ARIMA, using error metrics like Root Mean Square Error (RMSE) and Mean Absolute Error (MAE).

Once the model is optimized, it will be deployed as an independent RESTful microservice using a lightweight Python framework such as FastAPI. This architecture ensures that the computationally intensive model operations are isolated from the web application layer, enhancing modularity and scalability. The Next.js application will then communicate with the ML microservice through secure API endpoints, sending user inputs (such as selected stock tickers and prediction periods) and receiving the predicted price data in return.

The frontend application, developed in Next.js, will provide users with a visually appealing, interactive, and responsive interface. It will feature secure user authentication, stock ticker search functionality, and data visualization dashboards that display both historical and predicted price trends through dynamic charts. The interface will employ modern charting libraries such as Chart.js or Recharts to render the forecasts in real-time, enabling users to observe performance metrics and market patterns intuitively.

To ensure performance and scalability, both the frontend and backend microservices will be containerized using Docker. This will enable seamless deployment to cloud platforms such as Vercel (for Next.js) and AWS or (for the Python API). This setup follows an MLOps approach, ensuring smooth model versioning, consistent deployment, and easy maintenance of the system in production environments.

Furthermore, the project will include a comprehensive testing and validation phase, covering functional testing, integration testing and performance testing responsiveness

under load. This phase ensures that the application operates reliably under real-world conditions and provides accurate predictions to end-users.

From a usability perspective, the platform will be optimized for both desktop and mobile devices, ensuring accessibility for users with different levels of technical expertise. The web interface will prioritize clarity, speed, and simplicity to encourage adoption by investors, students, and researchers interested in data-driven financial insights.

Out of Scope

To maintain a realistic and focused development plan, certain features are excluded from this project's scope. These include real-time trading execution, integration with brokerage APIs, portfolio management, and macroeconomic factor analysis such as sentiment or news-based forecasting. While these features are valuable for future extensions, the current project is centered on forecasting, visualization, and system integration rather than full financial automation.

2.3 Project Keywords

Five keywords:
Long Short-Term Memory (LSTM)
Time Series Forecasting
Microservice Architecture
Financial Technology (FinTech)

Chapter 03: Research Gap

3.1 Review of Existing Solutions and Gaps

Methodological Gap (Modelling Accuracy):

Several studies, such as Ariyo, Adewumi and Ayo (2014) and Sunki et al. (2024), have applied traditional statistical models like ARIMA for stock price forecasting. Although these models are effective for linear and stationary datasets, they struggle to capture the complex, non-linear, and highly volatile behaviour of real-world financial data. More recent works, including Wu, Chen and Ding (2023), Li et al. (2023), and Yu (2025), demonstrate that deep learning approaches—particularly Long Short-Term Memory (LSTM) networks—can better handle temporal dependencies and dynamic relationships in time-series data. However, most existing implementations are confined to experimental research contexts and lack the scalability and robustness required for real-time or production-level forecasting systems.

Technological Integration Gap (Deployment and Accessibility):

Existing research efforts focus primarily on developing and testing models within Python-based environments such as TensorFlow or Keras (AI Stock Market Prediction with Python and LSTM, 2025). However, these implementations rarely explore integration with modern, interactive web frameworks. There remains a significant gap in solutions that seamlessly combine high-performance AI models with web technologies such as Next.js to provide real-time prediction services. This disconnect limits accessibility and usability for end-users who require dynamic and web-based financial forecasting tools.

Performance and Scalability Gap (System Architecture):

While studies such as Li et al. (2023) and Sunki et al. (2024) have demonstrated the predictive accuracy of deep learning models, many existing systems adopt monolithic or locally hosted architectures. These setups are computationally demanding and not designed for scalable deployment. There is a clear absence of microservice-based solutions that decouple the machine learning inference (Python) from the web interface (Next.js), enabling efficient communication and load distribution. Addressing this gap is essential for creating responsive, scalable, and cloud-ready forecasting systems.

3.2 Justification of Project Differentiation

This project distinguishes itself by integrating deep learning–based stock forecasting with a modern, scalable full-stack architecture. Building upon the strengths of LSTM models identified by Yu (2025) and Wu, Chen and Ding (2023), the system implements a microservice-based design that connects a Python/TensorFlow inference engine with a Next.js front-end interface. This combination ensures both predictive accuracy and a responsive, user-friendly interface for real-time forecasting.

Unlike prior research, which focuses mainly on algorithmic development, this project demonstrates an end-to-end MLOps workflow, encompassing model training, deployment, and web integration. The result is a production-ready, high-performance forecasting system that bridges the gap between academic AI models and practical web-based applications for financial analysis.

Chapter 04: Requirements Analysis

4.1 Functional Requirements

Requirement	Description
FR1 (Authentication)	The system shall allow users to register and securely log in.
FR2 (Prediction Request)	The system shall allow a user to input a stock ticker and a prediction horizon (e.g., 7 days).
FR3 (Visualization)	The Next.js frontend shall display the historical stock data and the projected forecast via an interactive chart library.
FR4 (Microservice Call)	The Next.js backend (API route) shall send input features to the Python ML microservice via a secure HTTP request to obtain the prediction result.

4.2 Non-Functional Requirements

Requirement	Description
NFR1 (Performance)	The complete prediction workflow (request to chart render) shall take less than 8 seconds to execute.
NFR2 (Security)	All user credentials and API communication must be encrypted (HTTPS and JWTs).
NFR3 (Scalability)	The application must be deployable as two separate, containerized services (Next.js and Python) to facilitate independent scaling (Microservice Strategy).
NFR4 (Usability)	The frontend application shall be fully responsive and optimized for both desktop and mobile devices.

4.3 Special Knowledge / Devices

- Special Knowledge: LSTM Deep Learning Architecture; Time-Series Data Preprocessing; Next.js Server-Side Rendering (SSR) and API Routes; Microservice Deployment (Docker/Containerization); Flask/FastAPI REST API Design.
- Special Devices: High-performance personal computer (min 16GB RAM) suitable for model training and local development of the dual-service stack.

Chapter 05: Finance

The following budget estimates the cost required to successfully complete the project.

Item	Description	Estimated Cost (LKR)
Cloud Hosting	1-year subscription for hosting the Next.js app and the Python ML Microservice	20,000
Domain Name	1-year domain registration.	5000
Electricity/Internet	Indirect operational costs for the development phase.	5000
Total Estimated Cost		30,000 LKR

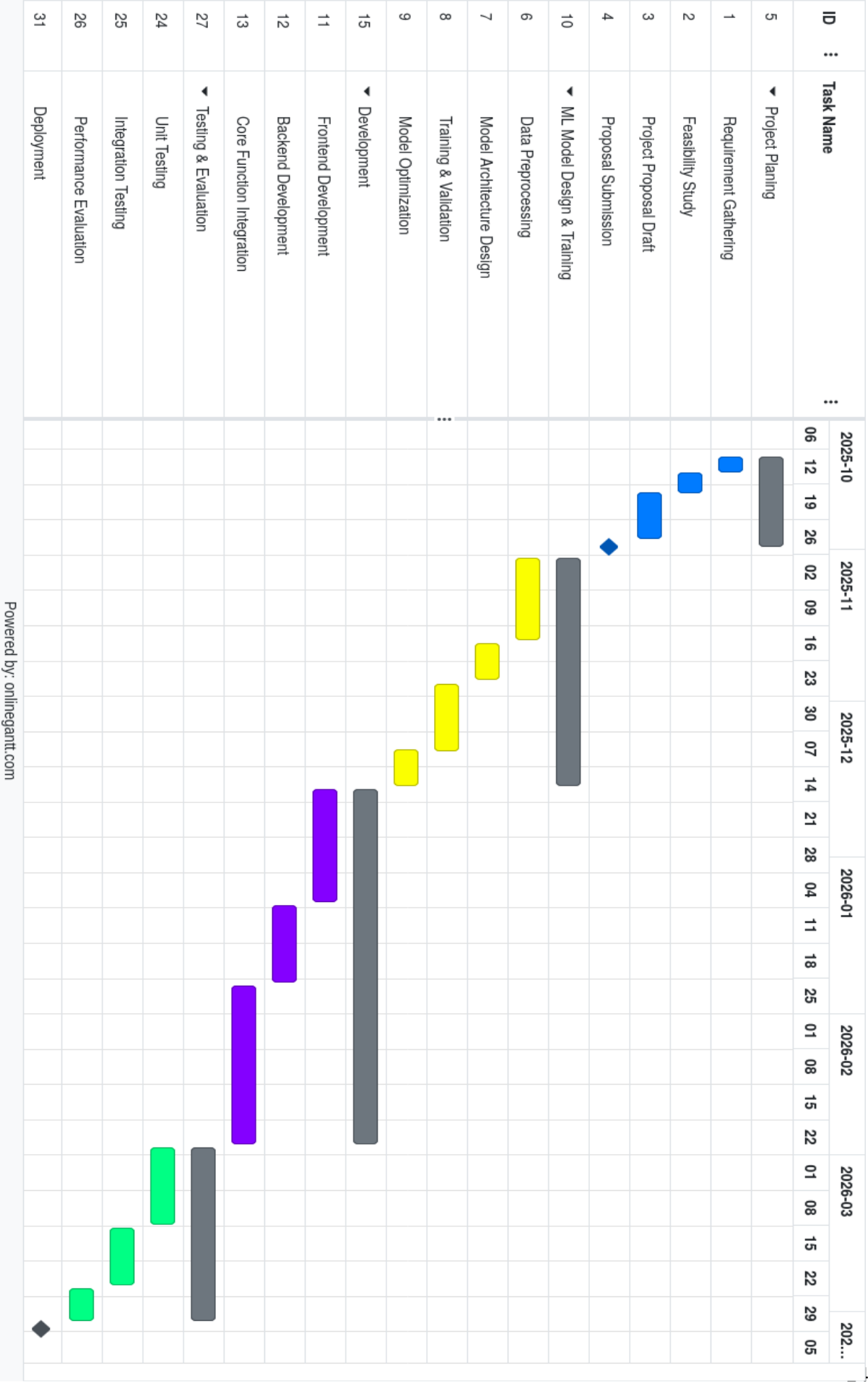
Chapter 06: External Organizations

External Entity	Role/Contribution
A Practicing Financial Analyst	Provides domain expertise for validating the utility of the technical indicators used and evaluating the real-world implications of the forecasting results.
Financial Data API Provider	Provides the necessary high-frequency, reliable financial data for real-time model training and inference.
Cloud Service Provider (e.g., Vercel, AWS)	Provides the platform for production deployment, enabling demonstration of MLOps and microservice containerization.

Chapter 07: Time Frame

The overall project time plan, presented from the beginning to the final report submission, is structured as follows.

Phase	Duration (Month)	Duration (Weeks)	Deliverable
Requirement & Research	October	Week 1–2	Project Proposal Submission (Current Document)
ML Model Design & Training	October-November	Week 3–6	Optimized LSTM Model (Trained, Saved weights) and Python ML Service API (Flask/FastAPI).
Next.js & Database Setup	December-January	Week 7–10	Next.js Project Initialization, Authentication Logic, and MongoDB Integration (User/Watchlist Schemas).
Microservice Integration	January	Week 11–13	Full API Bridge Integration between Next.js API Routes and Python ML Service. Basic End-to-End Prediction Test.
Frontend & Visualization	February	Week 14–16	Complete, Responsive Next.js Frontend with Interactive Charts (Prediction & Metrics Display).
Testing & Evaluation	February-March	Week 17–18	Model Comparison and Evaluation Report (RMSE/MAE comparison) and System Load Test Report.
Final Documentation	March	Week 19–20	Final Report Submission and Presentation Preparation.



Chapter 08: Referencing

Indian stock market prediction using artificial neural networks on tick data Published: 21 March 2019. Available at <https://jfin-swufe.springeropen.com/articles/10.1186/s40854-019-0131-7> (Accessed on 29th October 2025)

AI Stock Market Prediction with Python and LSTM." (2025). Artificial Intelligence in Plain English. Available at <https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf> Accessed on 29th October 2025)

Ariyo, A. A., Adewumi, A. O., & Ayo, C. K. (2014). "Stock Price Prediction Using the ARIMA Model." In Proceedings of the 2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation. Available at <https://ieeexplore.ieee.org/document/7046047> (Accessed on 29th October 2025)

Sunki, A., et al. (2024). "Time series forecasting of stock market using ARIMA, LSTM and FB prophet." MATEC Web of Conferences. Available at https://www.researchgate.net/publication/379050041_Time_series_forecasting_of_stock_market_using_ARIMA_LSTM_and_FB_prophet (Accessed on 29th October 2025)

Wu, H., Chen, S., & Ding, Y. (2023). "Comparison of ARIMA and LSTM for Stock Price Prediction." Financial Engineering and Risk Management. Available at <https://www.scirp.org/reference/referencespapers?referenceid=3830243> Accessed on 29th October 2025)

Yu, Y. (2025). "LSTM-Based Time Series Prediction Model: A Case Study with YFinance Stock Data." ITM Web of Conferences. Available at https://www.researchgate.net/publication/388315149_LSTM-Based_Time_Series_Prediction_Model_A_Case_Study_with_YFinance_Stock_Data (Accessed on 29th October 2025)

Li, Z., et al. (2023). "Stock Market Analysis and Prediction Using LSTM: A Case Study on Technology Stocks." Innovations in Applied Engineering and Technology. Available at https://www.researchgate.net/publication/379811995_Stock_Market_Analysis_and_Prediction_Using_LSTM_A_Case_Study_on_Technology_Stocks (Accessed on 29th October 2025)

<https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf>

